

abstraction torch

September 9, 2025

1 Imports and Setup

First, we import the necessary libraries. We'll use torch for all numerical computations, k3d for 3D visualization, and ipywidgets for the interactive editor.

```
[1]: import json
import os
import k3d
import torch
import ipywidgets as widgets
from ipywidgets import HBox, VBox, Layout
from IPython.display import display, clear_output
```

2 Configuration

We define a dictionary to map semantic labels (represented by integers) to specific colors for visualization. This helps in visually distinguishing different parts of a reconstructed shape. A default color is set for any unknown labels.

```
[2]: # A mapping from integer labels to hex color codes for visualization
LABEL_COLORS = {
    0: 0xFF0000, # red
    1: 0x00FF00, # green
    2: 0x0000FF, # blue
    3: 0xFFFF00, # yellow
    -1: 0x808080 # gray (Unknown)
}
```

3 PyTorch-based Rotation Utilities

The original code used `scipy.spatial.transform.Rotation`, which is based on NumPy. To make this project fully compatible with PyTorch, we need to replicate the necessary rotation functionalities (quaternion-to-matrix, matrix-to-quaternion, etc.) using torch operations. These utility functions will be crucial for calculating the orientation of the boxes.

```
[3]:
```

```

def quat_to_matrix(quat: torch.Tensor) -> torch.Tensor:
    """Converts a quaternion from (x, y, z, w) format to a 3x3 rotation matrix.
    ↪ """
    x, y, z, w = quat
    # Pre-calculate reused terms
    x2, y2, z2 = x * x, y * y, z * z
    xy, xz, yz = x * y, x * z, y * z
    wx, wy, wz = w * x, w * y, w * z

    # Create the rotation matrix
    return torch.tensor([
        [1 - 2 * (y2 + z2), 2 * (xy - wz), 2 * (xz + wy)],
        [2 * (xy + wz), 1 - 2 * (x2 + z2), 2 * (yz - wx)],
        [2 * (xz - wy), 2 * (yz + wx), 1 - 2 * (x2 + y2)]
    ], dtype=quat.dtype, device=quat.device)

def matrix_to_quat(matrix: torch.Tensor) -> torch.Tensor:
    """Converts a 3x3 rotation matrix to a quaternion in (x, y, z, w) format."""
    trace = matrix[0, 0] + matrix[1, 1] + matrix[2, 2]
    if trace > 0:
        s = 0.5 / torch.sqrt(trace + 1.0)
        w = 0.25 / s
        x = (matrix[2, 1] - matrix[1, 2]) * s
        y = (matrix[0, 2] - matrix[2, 0]) * s
        z = (matrix[1, 0] - matrix[0, 1]) * s
    else:
        if matrix[0, 0] > matrix[1, 1] and matrix[0, 0] > matrix[2, 2]:
            s = 2.0 * torch.sqrt(1.0 + matrix[0, 0] - matrix[1, 1] - matrix[2, 1]
            ↪ 2])
            w = (matrix[2, 1] - matrix[1, 2]) / s
            x = 0.25 * s
            y = (matrix[0, 1] + matrix[1, 0]) / s
            z = (matrix[0, 2] + matrix[2, 0]) / s
        elif matrix[1, 1] > matrix[2, 2]:
            s = 2.0 * torch.sqrt(1.0 + matrix[1, 1] - matrix[0, 0] - matrix[2, 1]
            ↪ 2])
            w = (matrix[0, 2] - matrix[2, 0]) / s
            x = (matrix[0, 1] + matrix[1, 0]) / s
            y = 0.25 * s
            z = (matrix[1, 2] + matrix[2, 1]) / s
        else:
            s = 2.0 * torch.sqrt(1.0 + matrix[2, 2] - matrix[0, 0] - matrix[1, 1]
            ↪ 1])
            w = (matrix[1, 0] - matrix[0, 1]) / s
            x = (matrix[0, 2] + matrix[2, 0]) / s
            y = (matrix[1, 2] + matrix[2, 1]) / s
            z = 0.25 * s

```

```

        return torch.tensor([x, y, z, w], dtype=matrix.dtype, device=matrix.device)

def rotvec_to_matrix(rotvec: torch.Tensor) -> torch.Tensor:
    """Converts a rotation vector to a 3x3 rotation matrix using Rodrigues'
    ↪ formula."""
    angle = torch.linalg.norm(rotvec)
    if angle == 0:
        return torch.eye(3, dtype=rotvec.dtype, device=rotvec.device)

    axis = rotvec / angle
    k_x, k_y, k_z = axis

    K = torch.tensor([
        [0, -k_z, k_y],
        [k_z, 0, -k_x],
        [-k_y, k_x, 0]
    ], dtype=rotvec.dtype, device=rotvec.device)

    I = torch.eye(3, dtype=rotvec.dtype, device=rotvec.device)
    # Rodrigues' rotation formula
    R = I + torch.sin(angle) * K + (1 - torch.cos(angle)) * (K @ K)
    return R

```

4 DSL Class Definitions

Here we define the classes that represent the nodes in our DSL tree. Each class corresponds to a specific shape primitive or operation. All numerical attributes (like center, dimensions, etc.) are stored as PyTorch tensors.

- Box: The basic building block, representing a cuboid in 3D space.
- Union: A node that combines two sub-trees into a single object.
- Symmetry: A base class for symmetry operations.
- SymRef, SymRot, SymTrans: Specific symmetry types for reflection, rotation, and translation, respectively.

```

[4]: class Box:
    def __init__(self, label: int, center, dims, quaternion):
        self.label = label
        self.center = torch.tensor(center, dtype=torch.float32)
        self.lengths = torch.tensor(dims, dtype=torch.float32)
        self.quaternion = torch.tensor(quaternion, dtype=torch.float32)

    def __str__(self):
        center_str = torch.round(self.center, decimals=2)
        lengths_str = torch.round(self.lengths, decimals=2)
        quat_str = torch.round(self.quaternion, decimals=2)

```

```

        return f"Box(label_id={self.label}, center={center_str.tolist()},  

↳dims={lengths_str.tolist()}, quat={quat_str.tolist()})"

class Union:
    def __init__(self, left, right):
        self.left = left
        self.right = right

    def __str__(self):
        return "Union"

class Symmetry:
    def __init__(self, child):
        self.child = child

class SymRef(Symmetry):
    def __init__(self, child, plane_normal, point_on_plane):
        super().__init__(child)
        self.plane = torch.tensor(plane_normal, dtype=torch.float32)
        self.point_on_plane = torch.tensor(point_on_plane, dtype=torch.float32)

    def __str__(self):
        plane_vec = torch.round(self.plane, decimals=2)
        point_vec = torch.round(self.point_on_plane, decimals=2)
        return f"Symmetry(Reflection) across plane with normal={plane_vec.  

↳tolist()} at point={point_vec.tolist()}"

class SymRot(Symmetry):
    def __init__(self, child, axis, center, n_fold: int):
        super().__init__(child)
        self.axis = torch.tensor(axis, dtype=torch.float32)
        self.center = torch.tensor(center, dtype=torch.float32)
        self.n = n_fold

    def __str__(self):
        axis_vec = torch.round(self.axis, decimals=2)
        center_vec = torch.round(self.center, decimals=2)
        return f"Symmetry(Rotation) of {self.n}-fold around axis={axis_vec.  

↳tolist()} at center={center_vec.tolist()}"

class SymTrans(Symmetry):
    def __init__(self, child, end_point, n_fold: int):
        super().__init__(child)
        self.end_point = torch.tensor(end_point, dtype=torch.float32)
        self.n = n_fold

    def __str__(self):

```

```

        end_point_vec = torch.round(self.end_point, decimals=2)
        return f"Symmetry(Translation) of {self.n} items towards_␣
↪end_point={end_point_vec.tolist()}"

```

5 JSON to DSL Parser

These functions handle the conversion from a JSON string to our DSL object tree. The `dict_to_dsl` function recursively traverses the dictionary loaded from JSON and instantiates the appropriate class for each node.

```

[5]: def dict_to_dsl(node_dict):
    """
    Recursively traverses a dictionary and converts it into the corresponding
    DSL object tree (Box, Union, SymRef, etc.).
    """
    if not isinstance(node_dict, dict):
        raise TypeError("Input must be a dictionary.")

    node_type = node_dict.get("type")

    if node_type == "Box":
        return Box(
            label=node_dict["label"],
            center=node_dict["center"],
            dims=node_dict["lengths"],
            quaternion=node_dict["quaternion"]
        )
    elif node_type == "Union":
        left_child = dict_to_dsl(node_dict["left"])
        right_child = dict_to_dsl(node_dict["right"])
        return Union(left=left_child, right=right_child)
    elif node_type == "SymRef":
        child_obj = dict_to_dsl(node_dict["child"])
        return SymRef(child=child_obj, plane_normal=node_dict["plane_normal"],␣
↪point_on_plane=node_dict["point_on_plane"])
    elif node_type == "SymRot":
        child_obj = dict_to_dsl(node_dict["child"])
        return SymRot(child=child_obj, axis=node_dict["axis"],␣
↪center=node_dict["center"], n_fold=node_dict["n_fold"])
    elif node_type == "SymTrans":
        child_obj = dict_to_dsl(node_dict["child"])
        return SymTrans(child=child_obj, end_point=node_dict["end_point"],␣
↪n_fold=node_dict["n_fold"])
    else:
        raise ValueError(f"Unknown node type found in JSON: '{node_type}'")

```

```
def parse_json_to_dsl(json_string):
    """Wrapper function to parse a JSON string into a DSL object tree."""
    data_dictionary = json.loads(json_string)
    return dict_to_dsl(data_dictionary)
```

6 DSL Tree Printing Utility

To better understand the structure of a parsed shape, these helper functions print the DSL tree in a readable, indented format.

```
[6]: def _print_dsl_recursive(node, prefix="", is_last=True):
    """Helper function to recursively print the tree."""
    # Define the connectors for the tree structure
    connector = "  " if is_last else "   "
    print(prefix + connector + str(node))

    # Adjust the prefix for child nodes
    child_prefix = prefix + ("    " if is_last else "   ")

    # Recurse into the children of the node
    if isinstance(node, Union):
        # A Union has two children, handle them with correct connectors
        _print_dsl_recursive(node.left, child_prefix, is_last=False)
        _print_dsl_recursive(node.right, child_prefix, is_last=True)
    elif isinstance(node, Symmetry):
        # Symmetries have only one child
        _print_dsl_recursive(node.child, child_prefix, is_last=True)

def print_dsl(root_node):
    """Prints a readable, tree-like structure of the DSL object hierarchy."""
    print("\n" + "="*25)
    print("  DSL Object Structure")
    print("="*25)
    _print_dsl_recursive(root_node)
    print("="*25 + "\n")
```

7 DSL Tree Expansion

This is the core logic of the program. The `expand_dsl_tree` function recursively walks the DSL tree and “unrolls” all the symmetry operations. For example, a `SymRot` node with a 4-fold rotation will be expanded into four copies of its child, each rotated by the appropriate angle. The final output is a flat list of dictionaries, where each dictionary represents a single `Box` with its final position, size, and orientation.

All calculations are performed with PyTorch tensors.

```

[7]: def expand_dsl_tree(node):
    """
    Recursively traverses the DSL object tree and expands all symmetries to
    generate
    a final, flat list of box dictionaries with torch tensors.
    """
    if isinstance(node, Box):
        return [{'center': node.center, 'lengths': node.lengths, 'quaternion':
        node.quaternion, 'label_id': node.label}]

    elif isinstance(node, Union):
        return expand_dsl_tree(node.left) + expand_dsl_tree(node.right)

    elif isinstance(node, (SymRef, SymRot, SymTrans)):
        child_boxes, generated_boxes = expand_dsl_tree(node.child), []

        if isinstance(node, SymRef):
            plane_normal = node.plane / (torch.linalg.norm(node.plane) + 1e-8)
            point_on_plane = node.point_on_plane
            for box in child_boxes:
                reflected_box = box.copy()
                vec_to_plane = box['center'] - point_on_plane
                dist = torch.dot(vec_to_plane, plane_normal)
                reflected_box['center'] = box['center'] - 2 * dist *
                plane_normal

                # Reflect the rotation
                R_orig = quat_to_matrix(box['quaternion'])
                M_reflect = torch.eye(3) - 2 * torch.outer(plane_normal,
                plane_normal)
                R_new = M_reflect @ R_orig

                # Ensure it's a valid rotation matrix (no reflection)
                if torch.linalg.det(R_new) < 0:
                    R_new[:, 0] *= -1

                reflected_box['quaternion'] = matrix_to_quat(R_new)
                generated_boxes.append(reflected_box)

        elif isinstance(node, SymRot):
            axis = node.axis / (torch.linalg.norm(node.axis) + 1e-8)
            center = node.center
            for i in range(1, node.n):
                angle = 2 * torch.pi * i / node.n
                symmetry_rot_mat = rotvec_to_matrix(angle * axis)

                for box in child_boxes:

```

```

        rotated_box = box.copy()
        # Apply rotation to the center
        rotated_box['center'] = center + (symmetry_rot_mat @
↳(box['center'] - center))

        # Compose rotations
        original_rot_mat = quat_to_matrix(box['quaternion'])
        new_rot_mat = symmetry_rot_mat @ original_rot_mat
        rotated_box['quaternion'] = matrix_to_quat(new_rot_mat)
        generated_boxes.append(rotated_box)

    elif isinstance(node, SymTrans):
        if child_boxes:
            start_point = child_boxes[0]['center']
            if node.n > 1:
                step_vector = (node.end_point - start_point) / (node.n - 1)
                for i in range(1, node.n):
                    translation = i * step_vector
                    for box in child_boxes:
                        translated_box = box.copy()
                        translated_box['center'] = box['center'] +
↳translation

                        generated_boxes.append(translated_box)

        return child_boxes + generated_boxes

    return []

```

8 Visualization with k3d

This function takes the root of a DSL tree, expands it into a list of boxes, and then uses k3d to render each box as a mesh in a 3D plot. Note that while all calculations use torch, the k3d.mesh function expects NumPy arrays as input, so we call .cpu().numpy() at the very last moment before plotting.

```

[8]: def plot_dsl_with_k3d(dsl_root_node):
    """Top-level function to expand a DSL tree and plot it using k3d."""
    print("Expanding DSL tree for visualization...")
    final_boxes = expand_dsl_tree(dsl_root_node)
    print(f"Found {len(final_boxes)} total boxes after expansion.")

    plot = k3d.plot(name="Reconstructed DSL Shape Visualization")

    for box in final_boxes:
        center, lengths, quaternion, label_id = box["center"], box["lengths"],
↳box["quaternion"], box.get("label_id", -1)

```

```

rotation_matrix = quat_to_matrix(quaternion)

# Calculate the 8 corners of the oriented box
d_vectors = [col * length / 2 for col, length in zip(rotation_matrix.T,
↳lengths)]
d1, d2, d3 = d_vectors[0], d_vectors[1], d_vectors[2]

# THE CORRECTED LINE: The definition of the 8 unique corners is now
↳correct.
corners = torch.stack([
    center-d1-d2-d3, center-d1+d2-d3, center+d1-d2-d3, center+d1+d2-d3,
    center-d1-d2+d3, center-d1+d2+d3, center+d1-d2+d3, center+d1+d2+d3
])

# Define the 12 triangular faces of the box
faces = torch.tensor([
    [0,1,3], [0,3,2], [4,6,7], [4,7,5], [0,2,6], [0,6,4],
    [1,5,7], [1,7,3], [0,4,5], [0,5,1], [2,3,7], [2,7,6]
], dtype=torch.uint32)

color = LABEL_COLORS.get(label_id, LABEL_COLORS[-1])
# Convert to numpy ONLY for plotting
plot += k3d.mesh(corners.cpu().numpy(), faces.cpu().numpy(),
↳color=color)

plot.display()

```

9 Example: Loading an External JSON File

This cell shows how to load a shape definition from a JSON file.

```

[9]: import random

random.seed(10)

# Define the path to the directory containing your JSON files
dataset_dir = "dataset/Chair"

# List all files in the directory and filter for those ending with .json
json_files = [f for f in os.listdir(dataset_dir) if f.endswith('.json')]

# Pick a random filename from the list and create the full path
random_filename = random.choice(json_files)
file_to_load = os.path.join(dataset_dir, random_filename)

print(f"Randomly selected: {file_to_load}")

```

```

# 1. Read the content of the randomly chosen JSON file
with open(file_to_load, 'r') as f:
    json_string = f.read()

# 2. Parse the JSON string into our live DSL object hierarchy
reconstructed_shape = parse_json_to_dsl(json_string)
print(f"Successfully parsed. Reconstructed object type is:␣
↪{type(reconstructed_shape)}")

# 3. Print the readable structure of the DSL tree
print_dsl(reconstructed_shape)

# 4. Pass the reconstructed object to the K3D plotting function
plot_dsl_with_k3d(reconstructed_shape)

```

Randomly selected: dataset/Chair\Chair_5210.json

Successfully parsed. Reconstructed object type is: <class '__main__.Union'>

```

=====
DSL Object Structure
=====

```

```

Union
  Union
    Box(label_id=1, center=[-0.009999999776482582, 0.009999999776482582,
0.18000000715255737], dims=[0.6899999976158142, 0.10000000149011612,
0.7300000190734863], quat=[-0.03999999910593033, -0.0, 0.0, 1.0])
    Union
      Box(label_id=2, center=[0.33000001311302185,
-0.36000001430511475, -0.23999999463558197], dims=[0.029999999329447746,
0.7300000190734863, 0.029999999329447746], quat=[0.23000000417232513,
0.23999999463558197, 0.10000000149011612, 0.9399999976158142])
      Union
        Box(label_id=2, center=[-0.3499999940395355,
-0.3499999940395355, -0.23999999463558197], dims=[0.029999999329447746, 0.75,
0.03999999910593033], quat=[0.23999999463558197, -0.0, -0.029999999329447746,
0.9700000286102295])
        Symmetry(Reflection) across plane with normal=[1.0, 0.0,
0.009999999776482582] at point=[-0.009999999776482582, -0.36000001430511475,
0.47999998927116394]
      Union
        Box(label_id=2, center=[-0.38999998569488525,
-0.36000001430511475, 0.47999998927116394], dims=[0.05000000074505806,
0.7900000214576721, 0.05999999865889549], quat=[-0.019999999552965164,
0.36000001430511475, -0.05000000074505806, 0.9300000071525574])
        Box(label_id=2, center=[-0.4099999964237213,
-0.7400000095367432, 0.07000000029802322], dims=[0.03999999910593033,
0.05000000074505806, 0.8899999856948853], quat=[0.0, -0.029999999329447746,

```

```

0.20000000298023224, 0.9800000190734863])
    Union
        Symmetry(Reflection) across plane with normal=[1.0,
-0.019999999552965164, 0.0] at point=[-0.009999999776482582,
0.27000001072883606, -0.27000001072883606]
        Box(label_id=0, center=[-0.28999999165534973,
0.27000001072883606, -0.27000001072883606], dims=[0.05000000074505806,
0.7200000286102295, 0.10999999940395355], quat=[-0.2199999988079071,
-0.009999999776482582, -0.019999999552965164, 0.9700000286102295])
    Union
        Box(label_id=0, center=[-0.009999999776482582,
0.28999999165534973, -0.2800000011920929], dims=[0.6200000047683716,
0.6800000071525574, 0.12999999523162842], quat=[-0.17000000178813934, 0.0, -0.0,
0.9900000095367432])
        Box(label_id=0, center=[-0.009999999776482582,
0.6000000238418579, -0.4099999964237213], dims=[0.5400000214576721,
0.029999999329447746, 0.03999999910593033], quat=[0.0, 0.0, 0.0, 1.0])
=====

```

Expanding DSL tree for visualization...

Found 11 total boxes after expansion.

Output()

10 Interactive GUI Editor

This final section creates an interactive editor for our DSL tree using ipywidgets. The NodeEditor class dynamically generates sliders and text boxes for all parameters in the tree. When you modify a value (e.g., the center of a box or the n_fold of a symmetry), the 3D plot on the right automatically updates. This provides a powerful way to experiment with the DSL and see the effects of parameter changes in real-time.

```

[10]: class NodeEditor:
    def __init__(self, root_node):
        self.root = root_node
        self.out = widgets.Output()
        self.tree_widget = self._make_node_widget(self.root)
        self.ui = HBox([
            VBox([widgets.Label("DSL Tree Editor"), self.tree_widget],
↪ layout=Layout(width="40%", border='1px solid lightgray', padding='10px',
↪ overflow='auto')),
            VBox([widgets.Label("3D Viewer"), self.out],
↪ layout=Layout(width="60%"))
        ])
        self._update_plot()

    def _make_array_editor(self, arr, label, callback):

```

```

        """Creates a row of FloatText widgets for editing a vector/tensor."""
        boxes = [widgets.FloatText(value=float(v), layout=Layout(width="80px"),
↪step=0.01) for v in arr]
        box = HBox([widgets.Label(label, layout=Layout(width="80px"))] + boxes)

    def on_change(change):
        new_vals = [b.value for b in boxes]
        callback(new_vals) # The callback will update the torch tensor
        self._update_plot()

    for b in boxes:
        b.observe(on_change, names="value")
    return box

def _make_node_widget(self, node):
    node_editors = []
    children_widgets = []
    title = "Node"

    # Dynamically create editors based on the node type
    if isinstance(node, Box):
        title = f"Box (Label: {node.label})"
        node_editors.append(self._make_array_editor(node.center, "center",
↪lambda v: setattr(node, 'center', torch.tensor(v, dtype=torch.float32))))
        node_editors.append(self._make_array_editor(node.lengths, "dims",
↪lambda v: setattr(node, 'lengths', torch.tensor(v, dtype=torch.float32))))
        node_editors.append(self._make_array_editor(node.quaternion,
↪"quat", lambda v: setattr(node, 'quaternion', torch.tensor(v, dtype=torch.
↪float32))))

    elif isinstance(node, Union):
        title = "Union"
        children_widgets.append(self._make_node_widget(node.left))
        children_widgets.append(self._make_node_widget(node.right))

    elif isinstance(node, SymRef):
        title = "Symmetry (Reflection)"
        node_editors.append(self._make_array_editor(node.plane, "plane",
↪lambda v: setattr(node, 'plane', torch.tensor(v, dtype=torch.float32))))
        node_editors.append(self._make_array_editor(node.point_on_plane,
↪"point", lambda v: setattr(node, 'point_on_plane', torch.tensor(v,
↪dtype=torch.float32))))
        children_widgets.append(self._make_node_widget(node.child))

    elif isinstance(node, SymRot):
        title = f"{node.n}-Fold Rotation"

```

```

        node_editors.append(self._make_array_editor(node.axis, "axis",
↪lambda v: setattr(node, 'axis', torch.tensor(v, dtype=torch.float32))))
        node_editors.append(self._make_array_editor(node.center, "center",
↪lambda v: setattr(node, 'center', torch.tensor(v, dtype=torch.float32))))
        n_editor = widgets.IntText(value=node.n, description="n-fold",
↪layout=Layout(width='auto'), style={'description_width': 'initial'})
        def on_n_change(change):
            node.n = change["new"]
            self._update_plot()
        n_editor.observe(on_n_change, names="value")
        node_editors.append(n_editor)
        children_widgets.append(self._make_node_widget(node.child))

    # Combine parameter editors and child widgets
    content = VBox(node_editors + children_widgets)
    accordion = widgets.Accordion(children=[content])
    accordion.set_title(0, title)
    return accordion

def _update_plot(self):
    """Redraws the 3D plot based on the current DSL tree state."""
    with self.out:
        clear_output(wait=True)
        plot = k3d.plot(name="Interactive Shape Viewer")
        final_boxes = expand_dsl_tree(self.root)
        for box in final_boxes:
            center, lengths, quat, label_id = box["center"],
↪box["lengths"], box["quaternion"], box.get("label_id", -1)
            rotation_matrix = quat_to_matrix(quat)
            d_vectors = [col * length / 2 for col, length in
↪zip(rotation_matrix.T, lengths)]
            d1, d2, d3 = d_vectors[0], d_vectors[1], d_vectors[2]
            corners = torch.stack([
                center-d1-d2-d3, center-d1+d2-d3, center+d1-d2-d3,
↪center+d1+d2-d3,
                center-d1-d2+d3, center-d1+d2+d3, center+d1-d2+d3,
↪center+d1+d2+d3
            ])
            faces = torch.
↪tensor([[0,1,3],[0,3,2],[4,6,7],[4,7,5],[0,2,6],[0,6,4],[1,5,7],[1,7,3],[0,4,5],[0,5,1],[2,
↪dtype=torch.uint32)
            color = LABEL_COLORS.get(label_id, LABEL_COLORS[-1])
            plot += k3d.mesh(corners.cpu().numpy(), faces.cpu().numpy(),
↪color=color)
        display(plot)

```

```

def show(self):
    """Displays the UI."""
    display(self.ui)

# --- Main execution for the interactive editor ---
# We can reuse the shape we parsed earlier
editor = NodeEditor(reconstructed_shape)
editor.show()

```

```

HBox(children=(VBox(children=(Label(value='DSL Tree Editor'),
↳Accordion(children=(VBox(children=(Accordion(chi...

```

11 Abstraction

```

[11]: import json
import os
import random
import torch
import torch.nn as nn
import torch.optim as optim
from tqdm import tqdm
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split

# --- Configuration & Setup ---

# Set to True to enable detailed debug prints throughout the forward passes.
ENABLE_DEBUG = False

# A mapping from integer labels to hex color codes for visualization
LABEL_COLORS = {
    0: 0xFF0000, # red
    1: 0x00FF00, # green
    2: 0x0000FF, # blue
    3: 0xFFFF00, # yellow
    -1: 0x808080 # gray (Unknown)
}

# Use a GPU if available, otherwise fall back to CPU
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {DEVICE}")

# Vocabulary for the Model
NODE_TYPE_TO_IDX = {
    "Box": 0, "Union": 1, "SymRef": 2, "SymRot": 3, "SymTrans": 4, "<EOS>": 5,
}
IDX_TO_NODE_TYPE = {v: k for k, v in NODE_TYPE_TO_IDX.items()}

```

```

# Number of parameters for each node type
PARAM_SIZE_MAP = {
    "Box": 11, "Union": 0, "SymRef": 6, "SymRot": 7, "SymTrans": 4
}

# Model Hyperparameters
HIDDEN_DIM = 128
LATENT_DIM = 64

```

Using device: cuda

```

[12]: # --- Vectorization and Devectorization Utilities ---

def vectorize_tree(node, device):
    """Converts a DSL tree to a nested dictionary of tensors, ready for the
    ↪model."""
    node_type = node.__class__.__name__
    vectorized_node = {
        "type": torch.tensor(NODE_TYPE_TO_IDX[node_type], dtype=torch.long,
        ↪device=device),
        "children": []
    }
    if isinstance(node, Box):
        params = [float(node.label)] + node.center.tolist() + \
            node.lengths.tolist() + node.quaternion.tolist()
        vectorized_node["params"] = torch.tensor(params, dtype=torch.float32,
        ↪device=device)
    elif isinstance(node, Union):
        vectorized_node["children"].append(vectorize_tree(node.left, device))
        vectorized_node["children"].append(vectorize_tree(node.right, device))
    elif isinstance(node, Symmetry):
        params = []
        if isinstance(node, SymRef):
            params = node.plane.tolist() + node.point_on_plane.tolist()
        elif isinstance(node, SymRot):
            params = node.axis.tolist() + node.center.tolist() + [float(node.n)]
        elif isinstance(node, SymTrans):
            params = node.end_point.tolist() + [float(node.n)]
        vectorized_node["params"] = torch.tensor(params, dtype=torch.float32,
        ↪device=device)
        vectorized_node["children"].append(vectorize_tree(node.child, device))

    return vectorized_node

def devectorize_tree(recon_node):

```

```

"""Converts a decoder's output dictionary back into a symbolic DSL object,
↳tree."""
    if recon_node is None: return None
    predicted_type_idx = torch.argmax(recon_node['type_logits']).item()
    predicted_type_str = IDX_TO_NODE_TYPE[predicted_type_idx]
    if predicted_type_str == "Box":
        p = recon_node['params'].cpu()
        return Box(label=int(p[0]), center=p[1:4], dims=p[4:7], quaternion=p[7:
↳11])
    elif predicted_type_str == "Union":
        if len(recon_node.get('children', [])) < 2: return None
        left_child = devectorize_tree(recon_node['children'][0])
        right_child = devectorize_tree(recon_node['children'][1])
        if left_child is None or right_child is None: return None
        return Union(left=left_child, right=right_child)
    elif predicted_type_str in ["SymRef", "SymRot", "SymTrans"]:
        if len(recon_node.get('children', [])) < 1: return None
        child = devectorize_tree(recon_node['children'][0])
        if child is None: return None
        p = recon_node['params'].cpu()
        if predicted_type_str == "SymRef":
            return SymRef(child=child, plane_normal=p[0:3], point_on_plane=p[3:
↳6])
        elif predicted_type_str == "SymRot":
            return SymRot(child=child, axis=p[0:3], center=p[3:6],
↳n_fold=int(p[6]))
        elif predicted_type_str == "SymTrans":
            return SymTrans(child=child, end_point=p[0:3], n_fold=int(p[3]))
    return None

def devectorize_tree_debug(recon_node, depth=0):
    """A debug version of devectorize_tree that prints step-by-step logic."""
    indent = " " * depth
    if recon_node is None:
        print(f"{indent}[DEBUG] Input node is None. Branch failed.")
        return None
    predicted_type_idx = torch.argmax(recon_node['type_logits']).item()
    predicted_type_str = IDX_TO_NODE_TYPE[predicted_type_idx]
    print(f"{indent}[DEBUG] Model PREDICTED node type: {predicted_type_str}")
    if predicted_type_str == "Box":
        print(f"{indent}[DEBUG] SUCCESS: Reached a valid leaf node (Box).")
        p = recon_node['params'].cpu()
        return Box(label=int(p[0]), center=p[1:4], dims=p[4:7], quaternion=p[7:
↳11])
    elif predicted_type_str == "Union":
        num_children = len(recon_node.get('children', []))
        if num_children < 2:

```

```

        print(f"{indent}[DEBUG]    FAIL: Predicted Union but found only_
↪{num_children} children. Expected 2.")
        return None
    print(f"{indent}[DEBUG] Recursing into 2 children for Union...")
    left_child = devectorize_tree_debug(recon_node['children'][0], depth +_
↪1)
    right_child = devectorize_tree_debug(recon_node['children'][1], depth +_
↪1)
    if left_child is None or right_child is None:
        print(f"{indent}[DEBUG]    FAIL: A child of the Union failed to_
↪reconstruct.")
        return None
    print(f"{indent}[DEBUG] SUCCESS: Reconstructed Union with 2 children.")
    return Union(left=left_child, right=right_child)
elif predicted_type_str in ["SymRef", "SymRot", "SymTrans"]:
    num_children = len(recon_node.get('children', []))
    if num_children < 1:
        print(f"{indent}[DEBUG]    FAIL: Predicted Symmetry but found_
↪{num_children} children. Expected 1.")
        return None
    print(f"{indent}[DEBUG] Recursing into 1 child for Symmetry...")
    child = devectorize_tree_debug(recon_node['children'][0], depth + 1)
    if child is None:
        print(f"{indent}[DEBUG]    FAIL: The child of the Symmetry node_
↪failed to reconstruct.")
        return None
    p = recon_node['params'].cpu()
    print(f"{indent}[DEBUG] SUCCESS: Reconstructed {predicted_type_str}_
↪with 1 child.")
    if predicted_type_str == "SymRef":
        return SymRef(child=child, plane_normal=p[0:3], point_on_plane=p[3:
↪6])
    elif predicted_type_str == "SymRot":
        return SymRot(child=child, axis=p[0:3], center=p[3:6],_
↪n_fold=int(p[6]))
    elif predicted_type_str == "SymTrans":
        return SymTrans(child=child, end_point=p[0:3], n_fold=int(p[3]))
    else:
        print(f"{indent}[DEBUG]    FAIL: Predicted an invalid or unknown node_
↪type '{predicted_type_str}'.")
        return None

```

[13]: # --- Model Architecture ---

```

class BoxEncoder(nn.Module):
    def __init__(self, hidden_dim):

```

```

        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(PARAM_SIZE_MAP["Box"], hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim)
        )
    def forward(self, params):
        if ENABLE_DEBUG: print(f" [BoxEncoder] Input shape: {params.shape}")
        return self.net(params)

class CompositionEncoder(nn.Module):
    def __init__(self, hidden_dim):
        super().__init__()
        self.nets = nn.ModuleDict({
            "Union": nn.Linear(2 * hidden_dim, hidden_dim),
            "SymRef": nn.Linear(hidden_dim + PARAM_SIZE_MAP["SymRef"],
↪hidden_dim),
            "SymRot": nn.Linear(hidden_dim + PARAM_SIZE_MAP["SymRot"],
↪hidden_dim),
            "SymTrans": nn.Linear(hidden_dim + PARAM_SIZE_MAP["SymTrans"],
↪hidden_dim),
        })

    def forward(self, node_type_str, params, child_features):
        if ENABLE_DEBUG: print(f" [CompositionEncoder] Node Type:
↪{node_type_str}")
        combined_child_features = torch.cat(child_features, dim=-1)
        if ENABLE_DEBUG: print(f" [CompositionEncoder] Combined child features
↪shape: {combined_child_features.shape}")
        if params is not None:
            if ENABLE_DEBUG: print(f" [CompositionEncoder] Param shape:
↪{params.shape}")
            model_input = torch.cat([combined_child_features, params], dim=-1)
        else:
            model_input = combined_child_features
        if ENABLE_DEBUG: print(f" [CompositionEncoder] Final input shape for
↪'{node_type_str}' net: {model_input.shape}")
        return self.nets[node_type_str](model_input)

class TreeEncoder(nn.Module):
    def __init__(self, hidden_dim, latent_dim):
        super().__init__()
        self.box_encoder = BoxEncoder(hidden_dim)
        self.composition_encoder = CompositionEncoder(hidden_dim)
        self.root_to_latent = nn.Linear(hidden_dim, latent_dim)

```

```

def _forward_recursive(self, v_node):
    node_type_str = IDX_TO_NODE_TYPE[v_node["type"].item()]
    if ENABLE_DEBUG: print(f" [Encoder] Processing node: {node_type_str}")
    if node_type_str == "Box":
        return self.box_encoder(v_node["params"])
    child_features = [self._forward_recursive(c) for c in
↳v_node["children"]]
    params = v_node.get("params")
    return self.composition_encoder(node_type_str, params, child_features)

def forward(self, v_tree):
    if ENABLE_DEBUG: print("--- Starting ENCODER forward pass ---")
    root_hidden_vec = self._forward_recursive(v_tree)
    if ENABLE_DEBUG: print(f" [Encoder] Root hidden vector shape:
↳{root_hidden_vec.shape}")
    z = self.root_to_latent(root_hidden_vec)
    if ENABLE_DEBUG: print("--- Finished ENCODER forward pass ---")
    return z

class NodeDecoder(nn.Module):
    def __init__(self, hidden_dim):
        super().__init__()
        self.net = nn.Sequential(nn.Linear(hidden_dim, hidden_dim), nn.ReLU())
        self.type_head = nn.Linear(hidden_dim, len(NODE_TYPE_TO_IDX))
        self.param_head = nn.Linear(hidden_dim, max(PARAM_SIZE_MAP.values()))
        self.child_head = nn.Linear(hidden_dim, 2 * hidden_dim)
    def forward(self, parent_feature):
        if ENABLE_DEBUG: print(f" [NodeDecoder] Input parent feature shape:
↳{parent_feature.shape}")
        h = self.net(parent_feature)
        return self.type_head(h), self.param_head(h), self.child_head(h)

class TreeDecoder(nn.Module):
    def __init__(self, latent_dim, hidden_dim):
        super().__init__()
        self.latent_to_hidden = nn.Linear(latent_dim, hidden_dim)
        self.node_decoder = NodeDecoder(hidden_dim)
    def _forward_recursive(self, parent_feature, gt_node):
        type_logits, param_preds, child_features = self.
↳node_decoder(parent_feature)
        reconstructed_node = {
            "type_logits": type_logits,
            "params": param_preds,
            "children": []
        }
        if gt_node["children"]:

```

```

        child1_feature, child2_feature = torch.chunk(child_features, 2,
↳dim=-1)
        child_features_split = [child1_feature, child2_feature]
        for i, gt_child_node in enumerate(gt_node["children"]):
            reconstructed_child = self.
↳_forward_recursive(child_features_split[i], gt_child_node)
            reconstructed_node["children"].append(reconstructed_child)
        return reconstructed_node
    def forward(self, z, gt_tree):
        initial_hidden = self.latent_to_hidden(z)
        return self._forward_recursive(initial_hidden, gt_tree)

class RecursiveAutoencoder(nn.Module):
    def __init__(self, hidden_dim, latent_dim):
        super().__init__()
        self.encoder = TreeEncoder(hidden_dim, latent_dim)
        self.decoder = TreeDecoder(latent_dim, hidden_dim)
    def forward(self, v_tree):
        z = self.encoder(v_tree)
        reconstructed_tree = self.decoder(z, v_tree)
        return reconstructed_tree

```

[14]: # --- Loss Function ---

```

def calculate_tree_loss(recon_node, gt_node):
    """Recursively calculates the reconstruction loss for a tree."""
    loss_fn_type = nn.CrossEntropyLoss()
    loss = loss_fn_type(recon_node['type_logits'].unsqueeze(0), gt_node['type'].
↳unsqueeze(0))
    gt_node_type_str = IDX_TO_NODE_TYPE[gt_node["type"].item()]
    num_params = PARAM_SIZE_MAP.get(gt_node_type_str, 0)
    if num_params > 0:
        loss_fn_params = nn.MSELoss()
        predicted_params = recon_node['params'][:num_params]
        ground_truth_params = gt_node['params']
        loss += loss_fn_params(predicted_params, ground_truth_params)
    for i, recon_child in enumerate(recon_node['children']):
        gt_child = gt_node['children'][i]
        loss += calculate_tree_loss(recon_child, gt_child)
    return loss

# --- Training Loop and Plotting ---

def train_model(model, dataset, epochs=20, learning_rate=0.001):
    """Trains the Recursive Autoencoder."""
    model.train()
    optimizer = optim.Adam(model.parameters(), lr=learning_rate)

```

```

loss_history = []
print(f"Starting training for {epochs} epochs...")
for epoch in range(epochs):
    total_epoch_loss = 0.0
    pbar = tqdm(dataset, desc=f"Epoch [{epoch+1}/{epochs}]")
    for v_tree in pbar:
        optimizer.zero_grad()
        reconstructed_tree = model(v_tree)
        loss = calculate_tree_loss(reconstructed_tree, v_tree)
        loss.backward()
        optimizer.step()
        total_epoch_loss += loss.item()
    pbar.set_postfix({'loss': f'{loss.item():.4f}'})
    avg_loss = total_epoch_loss / len(dataset)
    loss_history.append(avg_loss)
    print(f"Epoch [{epoch+1}/{epochs}] complete. Average Loss: {avg_loss:.4f}")
print("Training finished.")
return loss_history

def plot_loss_chart(losses):
    """Generates and saves a chart of the training loss."""
    plt.figure(figsize=(10, 6))
    plt.plot(losses, label='Training Loss')
    plt.title('Recursive Autoencoder Training Loss')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.legend()
    plt.grid(True)
    plt.savefig('training_loss_chart.png')
    print("Loss chart saved to 'training_loss_chart.png'")

```

```

[29]: from tqdm import tqdm

# --- 1. Load and Prepare the Dataset ---
dataset_dir = "dataset/Chair"
if not os.path.exists(dataset_dir):
    print(f"Error: Dataset directory '{dataset_dir}' not found.")
    print("Please ensure you have the dataset folder correctly set up.")
else:
    all_json_files = [f for f in os.listdir(dataset_dir) if f.endswith('.json')]
    files_to_process = all_json_files[:500]
    full_dataset = []
    print(f"Loading and vectorizing {len(files_to_process)} shapes...")
    for filename in tqdm(files_to_process, desc="Loading and Vectorizing Chairs"):
        filepath = os.path.join(dataset_dir, filename)

```

```

    with open(filepath, 'r') as f:
        json_string = f.read()
        dsl_tree = parse_json_to_dsl(json_string)
        vectorized_tree = vectorize_tree(dsl_tree, DEVICE)
        full_dataset.append((dsl_tree, vectorized_tree))
    print("Dataset prepared.")

    # --- 2. Split Data and Train the Model ---
    train_dataset, test_dataset = train_test_split(full_dataset, test_size=0.2,
    ↪random_state=42)
    training_vectors = [v_tree for _, v_tree in train_dataset]
    autoencoder = RecursiveAutoencoder(hidden_dim=HIDDEN_DIM,
    ↪latent_dim=LATENT_DIM).to(DEVICE)
    loss_history = train_model(autoencoder, training_vectors, epochs=200,
    ↪learning_rate=0.001)

    # --- 3. Visualize Loss and Check Reconstruction ---
    plot_loss_chart(loss_history)
    print("\n" + "#" * 50)
    print("### VISUALIZING A RECONSTRUCTION FROM THE TEST SET ###")
    print("#" * 50 + "\n")
    original_dsl_shape, vectorized_shape = random.choice(test_dataset)
    autoencoder.eval()
    with torch.no_grad():
        reconstruction_output = autoencoder(vectorized_shape)
    print("--- Original Shape Tree ---")
    print_dsl(original_dsl_shape)
    print("\n--- STARTING RECONSTRUCTION WITH DEBUG TRACE ---\n")
    final_reconstructed_shape = devectorize_tree_debug(reconstruction_output)
    print("\n--- FINISHED RECONSTRUCTION ATTEMPT ---\n")
    print("\n--- Reconstructed Shape Tree (after training) ---")
    if final_reconstructed_shape:
        print_dsl(final_reconstructed_shape)
    else:
        print("Reconstruction failed (the model predicted an invalid or
    ↪incomplete tree).")

```

Loading and vectorizing 500 shapes...

Loading and Vectorizing Chairs: 100% | 500/500 [00:00<00:00, 749.11it/s]

Dataset prepared.

Starting training for 200 epochs...

Epoch [1/200]: 100% | 400/400 [00:04<00:00, 88.13it/s, loss=5.7634]

Epoch [1/200] complete. Average Loss: 11.0592

Epoch [2/200]: 100% | 400/400 [00:04<00:00, 87.04it/s, loss=6.0921]

Epoch [2/200] complete. Average Loss: 8.0537
Epoch [3/200]: 100%| | 400/400 [00:04<00:00, 89.45it/s, loss=5.6989]
Epoch [3/200] complete. Average Loss: 6.8231
Epoch [4/200]: 100%| | 400/400 [00:04<00:00, 89.47it/s, loss=4.0467]
Epoch [4/200] complete. Average Loss: 6.4856
Epoch [5/200]: 100%| | 400/400 [00:04<00:00, 90.71it/s, loss=3.3542]
Epoch [5/200] complete. Average Loss: 6.1017
Epoch [6/200]: 100%| | 400/400 [00:04<00:00, 89.99it/s, loss=1.4742]
Epoch [6/200] complete. Average Loss: 5.3129
Epoch [7/200]: 100%| | 400/400 [00:04<00:00, 89.96it/s, loss=3.0140]
Epoch [7/200] complete. Average Loss: 5.1687
Epoch [8/200]: 100%| | 400/400 [00:04<00:00, 91.33it/s, loss=4.5378]
Epoch [8/200] complete. Average Loss: 5.5140
Epoch [9/200]: 100%| | 400/400 [00:04<00:00, 90.14it/s, loss=6.1872]
Epoch [9/200] complete. Average Loss: 4.7480
Epoch [10/200]: 100%| | 400/400 [00:04<00:00, 91.46it/s, loss=4.7545]
Epoch [10/200] complete. Average Loss: 5.3550
Epoch [11/200]: 100%| | 400/400 [00:04<00:00, 90.49it/s, loss=1.7205]
Epoch [11/200] complete. Average Loss: 4.4873
Epoch [12/200]: 100%| | 400/400 [00:04<00:00, 91.22it/s, loss=1.7817]
Epoch [12/200] complete. Average Loss: 4.1743
Epoch [13/200]: 100%| | 400/400 [00:04<00:00, 90.32it/s, loss=2.4517]
Epoch [13/200] complete. Average Loss: 4.1224
Epoch [14/200]: 100%| | 400/400 [00:04<00:00, 91.56it/s, loss=1.6717]
Epoch [14/200] complete. Average Loss: 4.2712
Epoch [15/200]: 100%| | 400/400 [00:04<00:00, 89.35it/s, loss=3.0542]
Epoch [15/200] complete. Average Loss: 3.7927
Epoch [16/200]: 100%| | 400/400 [00:04<00:00, 88.87it/s, loss=2.7478]
Epoch [16/200] complete. Average Loss: 3.7958
Epoch [17/200]: 100%| | 400/400 [00:04<00:00, 91.33it/s, loss=1.4867]
Epoch [17/200] complete. Average Loss: 4.0258
Epoch [18/200]: 100%| | 400/400 [00:04<00:00, 91.54it/s, loss=4.4252]

Epoch [18/200] complete. Average Loss: 3.7702
Epoch [19/200]: 100%| | 400/400 [00:04<00:00, 91.37it/s, loss=1.7907]
Epoch [19/200] complete. Average Loss: 4.3498
Epoch [20/200]: 100%| | 400/400 [00:04<00:00, 89.95it/s, loss=2.2478]
Epoch [20/200] complete. Average Loss: 3.2933
Epoch [21/200]: 100%| | 400/400 [00:04<00:00, 89.74it/s, loss=1.3059]
Epoch [21/200] complete. Average Loss: 3.6276
Epoch [22/200]: 100%| | 400/400 [00:04<00:00, 89.95it/s, loss=9.4564]
Epoch [22/200] complete. Average Loss: 3.3841
Epoch [23/200]: 100%| | 400/400 [00:04<00:00, 89.54it/s, loss=0.9113]
Epoch [23/200] complete. Average Loss: 3.1758
Epoch [24/200]: 100%| | 400/400 [00:04<00:00, 89.03it/s, loss=3.0109]
Epoch [24/200] complete. Average Loss: 3.1027
Epoch [25/200]: 100%| | 400/400 [00:04<00:00, 89.24it/s, loss=4.9717]
Epoch [25/200] complete. Average Loss: 3.8585
Epoch [26/200]: 100%| | 400/400 [00:04<00:00, 88.69it/s, loss=4.1245]
Epoch [26/200] complete. Average Loss: 2.9629
Epoch [27/200]: 100%| | 400/400 [00:04<00:00, 89.68it/s, loss=1.0248]
Epoch [27/200] complete. Average Loss: 4.4533
Epoch [28/200]: 100%| | 400/400 [00:04<00:00, 89.22it/s, loss=1.2709]
Epoch [28/200] complete. Average Loss: 3.1707
Epoch [29/200]: 100%| | 400/400 [00:04<00:00, 90.04it/s, loss=0.7782]
Epoch [29/200] complete. Average Loss: 2.8186
Epoch [30/200]: 100%| | 400/400 [00:04<00:00, 89.19it/s, loss=3.7874]
Epoch [30/200] complete. Average Loss: 3.2290
Epoch [31/200]: 100%| | 400/400 [00:04<00:00, 88.65it/s, loss=1.5955]
Epoch [31/200] complete. Average Loss: 3.7162
Epoch [32/200]: 100%| | 400/400 [00:04<00:00, 89.90it/s, loss=4.0155]
Epoch [32/200] complete. Average Loss: 3.6240
Epoch [33/200]: 100%| | 400/400 [00:04<00:00, 89.71it/s, loss=2.3469]
Epoch [33/200] complete. Average Loss: 3.4369
Epoch [34/200]: 100%| | 400/400 [00:04<00:00, 88.09it/s, loss=2.3722]

Epoch [34/200] complete. Average Loss: 2.8817
Epoch [35/200]: 100%| | 400/400 [00:04<00:00, 89.38it/s, loss=7.9381]
Epoch [35/200] complete. Average Loss: 2.6426
Epoch [36/200]: 100%| | 400/400 [00:04<00:00, 89.47it/s, loss=0.8666]
Epoch [36/200] complete. Average Loss: 2.8252
Epoch [37/200]: 100%| | 400/400 [00:04<00:00, 90.14it/s, loss=1.9142]
Epoch [37/200] complete. Average Loss: 2.9823
Epoch [38/200]: 100%| | 400/400 [00:04<00:00, 88.60it/s, loss=0.8597]
Epoch [38/200] complete. Average Loss: 2.8482
Epoch [39/200]: 100%| | 400/400 [00:04<00:00, 88.49it/s, loss=0.4380]
Epoch [39/200] complete. Average Loss: 2.4830
Epoch [40/200]: 100%| | 400/400 [00:04<00:00, 89.70it/s, loss=5.1887]
Epoch [40/200] complete. Average Loss: 3.5602
Epoch [41/200]: 100%| | 400/400 [00:04<00:00, 88.39it/s, loss=6.7481]
Epoch [41/200] complete. Average Loss: 3.0894
Epoch [42/200]: 100%| | 400/400 [00:04<00:00, 90.70it/s, loss=0.7770]
Epoch [42/200] complete. Average Loss: 3.4824
Epoch [43/200]: 100%| | 400/400 [00:04<00:00, 89.59it/s, loss=4.6861]
Epoch [43/200] complete. Average Loss: 4.3400
Epoch [44/200]: 100%| | 400/400 [00:04<00:00, 88.51it/s, loss=0.4084]
Epoch [44/200] complete. Average Loss: 3.1355
Epoch [45/200]: 100%| | 400/400 [00:04<00:00, 90.04it/s, loss=0.6053]
Epoch [45/200] complete. Average Loss: 2.4298
Epoch [46/200]: 100%| | 400/400 [00:04<00:00, 89.22it/s, loss=4.2365]
Epoch [46/200] complete. Average Loss: 2.7936
Epoch [47/200]: 100%| | 400/400 [00:04<00:00, 89.67it/s, loss=0.4700]
Epoch [47/200] complete. Average Loss: 3.1132
Epoch [48/200]: 100%| | 400/400 [00:04<00:00, 88.27it/s, loss=3.5861]
Epoch [48/200] complete. Average Loss: 3.4487
Epoch [49/200]: 100%| | 400/400 [00:04<00:00, 88.37it/s, loss=0.4372]
Epoch [49/200] complete. Average Loss: 3.0630
Epoch [50/200]: 100%| | 400/400 [00:04<00:00, 89.96it/s, loss=1.9331]

Epoch [50/200] complete. Average Loss: 2.6765
Epoch [51/200]: 100%| | 400/400 [00:04<00:00, 88.24it/s, loss=0.8425]
Epoch [51/200] complete. Average Loss: 3.0155
Epoch [52/200]: 100%| | 400/400 [00:04<00:00, 90.56it/s, loss=4.0828]
Epoch [52/200] complete. Average Loss: 2.9627
Epoch [53/200]: 100%| | 400/400 [00:04<00:00, 89.71it/s, loss=0.4131]
Epoch [53/200] complete. Average Loss: 2.8766
Epoch [54/200]: 100%| | 400/400 [00:04<00:00, 89.73it/s, loss=0.4232]
Epoch [54/200] complete. Average Loss: 3.1156
Epoch [55/200]: 100%| | 400/400 [00:04<00:00, 89.18it/s, loss=0.4911]
Epoch [55/200] complete. Average Loss: 3.3987
Epoch [56/200]: 100%| | 400/400 [00:04<00:00, 89.19it/s, loss=3.5589]
Epoch [56/200] complete. Average Loss: 2.8983
Epoch [57/200]: 100%| | 400/400 [00:04<00:00, 90.56it/s, loss=1.8008]
Epoch [57/200] complete. Average Loss: 2.4533
Epoch [58/200]: 100%| | 400/400 [00:04<00:00, 89.26it/s, loss=12.0396]
Epoch [58/200] complete. Average Loss: 2.7903
Epoch [59/200]: 100%| | 400/400 [00:04<00:00, 89.84it/s, loss=1.0883]
Epoch [59/200] complete. Average Loss: 2.5579
Epoch [60/200]: 100%| | 400/400 [00:04<00:00, 89.53it/s, loss=6.4662]
Epoch [60/200] complete. Average Loss: 3.2330
Epoch [61/200]: 100%| | 400/400 [00:04<00:00, 91.08it/s, loss=10.5634]
Epoch [61/200] complete. Average Loss: 3.0207
Epoch [62/200]: 100%| | 400/400 [00:04<00:00, 90.87it/s, loss=0.2999]
Epoch [62/200] complete. Average Loss: 2.3184
Epoch [63/200]: 100%| | 400/400 [00:04<00:00, 89.24it/s, loss=1.5232]
Epoch [63/200] complete. Average Loss: 2.9927
Epoch [64/200]: 100%| | 400/400 [00:04<00:00, 88.87it/s, loss=0.4727]
Epoch [64/200] complete. Average Loss: 2.2520
Epoch [65/200]: 100%| | 400/400 [00:04<00:00, 89.25it/s, loss=0.3318]
Epoch [65/200] complete. Average Loss: 3.2711
Epoch [66/200]: 100%| | 400/400 [00:04<00:00, 90.38it/s, loss=0.9129]

Epoch [66/200] complete. Average Loss: 3.0286
Epoch [67/200]: 100%| | 400/400 [00:04<00:00, 89.00it/s, loss=1.0415]
Epoch [67/200] complete. Average Loss: 3.1053
Epoch [68/200]: 100%| | 400/400 [00:04<00:00, 89.65it/s, loss=0.9998]
Epoch [68/200] complete. Average Loss: 2.5101
Epoch [69/200]: 100%| | 400/400 [00:04<00:00, 90.01it/s, loss=0.3400]
Epoch [69/200] complete. Average Loss: 2.0853
Epoch [70/200]: 100%| | 400/400 [00:04<00:00, 88.11it/s, loss=6.0768]
Epoch [70/200] complete. Average Loss: 2.3207
Epoch [71/200]: 100%| | 400/400 [00:04<00:00, 89.24it/s, loss=8.4735]
Epoch [71/200] complete. Average Loss: 2.5312
Epoch [72/200]: 100%| | 400/400 [00:04<00:00, 88.95it/s, loss=2.5555]
Epoch [72/200] complete. Average Loss: 3.1372
Epoch [73/200]: 100%| | 400/400 [00:04<00:00, 89.71it/s, loss=2.8050]
Epoch [73/200] complete. Average Loss: 3.3724
Epoch [74/200]: 100%| | 400/400 [00:04<00:00, 90.20it/s, loss=2.4960]
Epoch [74/200] complete. Average Loss: 2.5035
Epoch [75/200]: 100%| | 400/400 [00:04<00:00, 90.00it/s, loss=0.4491]
Epoch [75/200] complete. Average Loss: 2.6928
Epoch [76/200]: 100%| | 400/400 [00:04<00:00, 90.86it/s, loss=0.8413]
Epoch [76/200] complete. Average Loss: 2.3288
Epoch [77/200]: 100%| | 400/400 [00:04<00:00, 89.31it/s, loss=4.4220]
Epoch [77/200] complete. Average Loss: 2.6013
Epoch [78/200]: 100%| | 400/400 [00:04<00:00, 89.77it/s, loss=5.8866]
Epoch [78/200] complete. Average Loss: 3.0000
Epoch [79/200]: 100%| | 400/400 [00:04<00:00, 89.83it/s, loss=1.8805]
Epoch [79/200] complete. Average Loss: 2.7775
Epoch [80/200]: 100%| | 400/400 [00:04<00:00, 89.07it/s, loss=0.3397]
Epoch [80/200] complete. Average Loss: 2.2999
Epoch [81/200]: 100%| | 400/400 [00:04<00:00, 90.09it/s, loss=1.9196]
Epoch [81/200] complete. Average Loss: 2.0777
Epoch [82/200]: 100%| | 400/400 [00:04<00:00, 89.64it/s, loss=0.5608]

Epoch [82/200] complete. Average Loss: 2.1923
Epoch [83/200]: 100%| | 400/400 [00:04<00:00, 90.24it/s, loss=0.9403]
Epoch [83/200] complete. Average Loss: 2.2924
Epoch [84/200]: 100%| | 400/400 [00:04<00:00, 88.54it/s, loss=4.9364]
Epoch [84/200] complete. Average Loss: 5.0081
Epoch [85/200]: 100%| | 400/400 [00:04<00:00, 89.95it/s, loss=0.6648]
Epoch [85/200] complete. Average Loss: 3.8304
Epoch [86/200]: 100%| | 400/400 [00:04<00:00, 88.98it/s, loss=0.6481]
Epoch [86/200] complete. Average Loss: 2.5423
Epoch [87/200]: 100%| | 400/400 [00:04<00:00, 89.09it/s, loss=0.3208]
Epoch [87/200] complete. Average Loss: 2.1143
Epoch [88/200]: 100%| | 400/400 [00:04<00:00, 89.37it/s, loss=1.6891]
Epoch [88/200] complete. Average Loss: 2.4174
Epoch [89/200]: 100%| | 400/400 [00:04<00:00, 89.24it/s, loss=2.3851]
Epoch [89/200] complete. Average Loss: 2.3760
Epoch [90/200]: 100%| | 400/400 [00:04<00:00, 90.29it/s, loss=16.3601]
Epoch [90/200] complete. Average Loss: 2.7518
Epoch [91/200]: 100%| | 400/400 [00:04<00:00, 89.25it/s, loss=2.7178]
Epoch [91/200] complete. Average Loss: 2.6654
Epoch [92/200]: 100%| | 400/400 [00:04<00:00, 89.95it/s, loss=0.6858]
Epoch [92/200] complete. Average Loss: 2.8036
Epoch [93/200]: 100%| | 400/400 [00:04<00:00, 91.40it/s, loss=1.4463]
Epoch [93/200] complete. Average Loss: 2.4055
Epoch [94/200]: 100%| | 400/400 [00:04<00:00, 89.86it/s, loss=1.6861]
Epoch [94/200] complete. Average Loss: 2.3624
Epoch [95/200]: 100%| | 400/400 [00:04<00:00, 89.83it/s, loss=0.7333]
Epoch [95/200] complete. Average Loss: 2.6626
Epoch [96/200]: 100%| | 400/400 [00:04<00:00, 89.08it/s, loss=0.5613]
Epoch [96/200] complete. Average Loss: 1.9444
Epoch [97/200]: 100%| | 400/400 [00:04<00:00, 89.14it/s, loss=0.4336]
Epoch [97/200] complete. Average Loss: 2.3930
Epoch [98/200]: 100%| | 400/400 [00:04<00:00, 90.56it/s, loss=8.0457]

Epoch [98/200] complete. Average Loss: 2.7541
Epoch [99/200]: 100%| | 400/400 [00:04<00:00, 89.15it/s, loss=0.4945]
Epoch [99/200] complete. Average Loss: 2.8215
Epoch [100/200]: 100%| | 400/400 [00:04<00:00, 89.30it/s, loss=1.8108]
Epoch [100/200] complete. Average Loss: 2.5181
Epoch [101/200]: 100%| | 400/400 [00:04<00:00, 89.66it/s, loss=10.8116]
Epoch [101/200] complete. Average Loss: 2.5927
Epoch [102/200]: 100%| | 400/400 [00:04<00:00, 90.12it/s, loss=1.4321]
Epoch [102/200] complete. Average Loss: 3.6788
Epoch [103/200]: 100%| | 400/400 [00:04<00:00, 89.93it/s, loss=0.5608]
Epoch [103/200] complete. Average Loss: 4.1474
Epoch [104/200]: 100%| | 400/400 [00:04<00:00, 88.15it/s, loss=1.4387]
Epoch [104/200] complete. Average Loss: 3.6539
Epoch [105/200]: 100%| | 400/400 [00:04<00:00, 89.27it/s, loss=0.2893]
Epoch [105/200] complete. Average Loss: 2.6886
Epoch [106/200]: 100%| | 400/400 [00:04<00:00, 89.79it/s, loss=0.3686]
Epoch [106/200] complete. Average Loss: 1.8944
Epoch [107/200]: 100%| | 400/400 [00:04<00:00, 88.47it/s, loss=1.9660]
Epoch [107/200] complete. Average Loss: 1.7342
Epoch [108/200]: 100%| | 400/400 [00:04<00:00, 89.46it/s, loss=15.5135]
Epoch [108/200] complete. Average Loss: 1.7188
Epoch [109/200]: 100%| | 400/400 [00:04<00:00, 90.24it/s, loss=2.7811]
Epoch [109/200] complete. Average Loss: 2.9370
Epoch [110/200]: 100%| | 400/400 [00:04<00:00, 89.35it/s, loss=0.6939]
Epoch [110/200] complete. Average Loss: 3.3170
Epoch [111/200]: 100%| | 400/400 [00:04<00:00, 90.39it/s, loss=0.4758]
Epoch [111/200] complete. Average Loss: 2.9764
Epoch [112/200]: 100%| | 400/400 [00:04<00:00, 89.26it/s, loss=0.5133]
Epoch [112/200] complete. Average Loss: 2.2026
Epoch [113/200]: 100%| | 400/400 [00:04<00:00, 90.22it/s, loss=0.3972]
Epoch [113/200] complete. Average Loss: 1.6620
Epoch [114/200]: 100%| | 400/400 [00:04<00:00, 90.35it/s, loss=2.8415]

Epoch [114/200] complete. Average Loss: 2.8663
Epoch [115/200]: 100%| | 400/400 [00:04<00:00, 90.29it/s, loss=2.2830]
Epoch [115/200] complete. Average Loss: 2.5464
Epoch [116/200]: 100%| | 400/400 [00:04<00:00, 90.80it/s, loss=0.7114]
Epoch [116/200] complete. Average Loss: 2.2931
Epoch [117/200]: 100%| | 400/400 [00:04<00:00, 89.01it/s, loss=3.6675]
Epoch [117/200] complete. Average Loss: 2.0937
Epoch [118/200]: 100%| | 400/400 [00:04<00:00, 90.57it/s, loss=0.2590]
Epoch [118/200] complete. Average Loss: 2.1544
Epoch [119/200]: 100%| | 400/400 [00:04<00:00, 88.55it/s, loss=2.2604]
Epoch [119/200] complete. Average Loss: 2.0037
Epoch [120/200]: 100%| | 400/400 [00:04<00:00, 88.75it/s, loss=1.7953]
Epoch [120/200] complete. Average Loss: 3.3491
Epoch [121/200]: 100%| | 400/400 [00:04<00:00, 89.12it/s, loss=0.8120]
Epoch [121/200] complete. Average Loss: 3.1927
Epoch [122/200]: 100%| | 400/400 [00:04<00:00, 89.03it/s, loss=1.0775]
Epoch [122/200] complete. Average Loss: 2.0278
Epoch [123/200]: 100%| | 400/400 [00:04<00:00, 89.80it/s, loss=0.4077]
Epoch [123/200] complete. Average Loss: 2.3936
Epoch [124/200]: 100%| | 400/400 [00:04<00:00, 90.05it/s, loss=0.4497]
Epoch [124/200] complete. Average Loss: 1.8750
Epoch [125/200]: 100%| | 400/400 [00:04<00:00, 90.03it/s, loss=0.3908]
Epoch [125/200] complete. Average Loss: 1.8039
Epoch [126/200]: 100%| | 400/400 [00:04<00:00, 89.15it/s, loss=0.9167]
Epoch [126/200] complete. Average Loss: 1.5621
Epoch [127/200]: 100%| | 400/400 [00:04<00:00, 89.00it/s, loss=0.5316]
Epoch [127/200] complete. Average Loss: 2.1350
Epoch [128/200]: 100%| | 400/400 [00:04<00:00, 88.59it/s, loss=5.2391]
Epoch [128/200] complete. Average Loss: 2.2795
Epoch [129/200]: 100%| | 400/400 [00:04<00:00, 89.08it/s, loss=3.4712]
Epoch [129/200] complete. Average Loss: 2.5383
Epoch [130/200]: 100%| | 400/400 [00:04<00:00, 91.19it/s, loss=0.9236]

Epoch [130/200] complete. Average Loss: 2.2921

Epoch [131/200]: 100%| | 400/400 [00:04<00:00, 88.83it/s, loss=1.3844]

Epoch [131/200] complete. Average Loss: 2.1994

Epoch [132/200]: 100%| | 400/400 [00:04<00:00, 89.11it/s, loss=0.2994]

Epoch [132/200] complete. Average Loss: 2.6587

Epoch [133/200]: 100%| | 400/400 [00:04<00:00, 89.71it/s, loss=0.2975]

Epoch [133/200] complete. Average Loss: 2.0560

Epoch [134/200]: 100%| | 400/400 [00:04<00:00, 88.85it/s, loss=2.5361]

Epoch [134/200] complete. Average Loss: 1.6653

Epoch [135/200]: 100%| | 400/400 [00:04<00:00, 90.10it/s, loss=4.9525]

Epoch [135/200] complete. Average Loss: 2.4152

Epoch [136/200]: 100%| | 400/400 [00:04<00:00, 88.78it/s, loss=0.5155]

Epoch [136/200] complete. Average Loss: 2.3042

Epoch [137/200]: 100%| | 400/400 [00:04<00:00, 88.23it/s, loss=1.2877]

Epoch [137/200] complete. Average Loss: 1.9625

Epoch [138/200]: 100%| | 400/400 [00:04<00:00, 89.85it/s, loss=0.6508]

Epoch [138/200] complete. Average Loss: 2.4601

Epoch [139/200]: 100%| | 400/400 [00:04<00:00, 89.95it/s, loss=3.1971]

Epoch [139/200] complete. Average Loss: 2.8651

Epoch [140/200]: 100%| | 400/400 [00:04<00:00, 90.51it/s, loss=0.8222]

Epoch [140/200] complete. Average Loss: 2.3143

Epoch [141/200]: 100%| | 400/400 [00:04<00:00, 89.37it/s, loss=0.7720]

Epoch [141/200] complete. Average Loss: 1.5673

Epoch [142/200]: 100%| | 400/400 [00:04<00:00, 89.44it/s, loss=10.0624]

Epoch [142/200] complete. Average Loss: 2.5240

Epoch [143/200]: 100%| | 400/400 [00:04<00:00, 90.23it/s, loss=2.0753]

Epoch [143/200] complete. Average Loss: 2.6999

Epoch [144/200]: 100%| | 400/400 [00:04<00:00, 89.51it/s, loss=0.6991]

Epoch [144/200] complete. Average Loss: 2.6085

Epoch [145/200]: 100%| | 400/400 [00:04<00:00, 90.15it/s, loss=0.4667]

Epoch [145/200] complete. Average Loss: 2.9075

Epoch [146/200]: 100%| | 400/400 [00:04<00:00, 90.02it/s, loss=8.2033]

Epoch [146/200] complete. Average Loss: 3.5889
Epoch [147/200]: 100%| | 400/400 [00:04<00:00, 89.60it/s, loss=4.2292]
Epoch [147/200] complete. Average Loss: 2.8276
Epoch [148/200]: 100%| | 400/400 [00:04<00:00, 90.36it/s, loss=2.8842]
Epoch [148/200] complete. Average Loss: 1.8750
Epoch [149/200]: 100%| | 400/400 [00:04<00:00, 88.83it/s, loss=0.7450]
Epoch [149/200] complete. Average Loss: 3.4826
Epoch [150/200]: 100%| | 400/400 [00:04<00:00, 90.37it/s, loss=0.5955]
Epoch [150/200] complete. Average Loss: 4.2300
Epoch [151/200]: 100%| | 400/400 [00:04<00:00, 89.48it/s, loss=0.4441]
Epoch [151/200] complete. Average Loss: 2.0708
Epoch [152/200]: 100%| | 400/400 [00:04<00:00, 89.04it/s, loss=0.6629]
Epoch [152/200] complete. Average Loss: 1.7725
Epoch [153/200]: 100%| | 400/400 [00:04<00:00, 89.48it/s, loss=0.3001]
Epoch [153/200] complete. Average Loss: 2.2506
Epoch [154/200]: 100%| | 400/400 [00:04<00:00, 88.78it/s, loss=5.8878]
Epoch [154/200] complete. Average Loss: 2.1464
Epoch [155/200]: 100%| | 400/400 [00:04<00:00, 89.89it/s, loss=6.3467]
Epoch [155/200] complete. Average Loss: 1.8169
Epoch [156/200]: 100%| | 400/400 [00:04<00:00, 89.80it/s, loss=4.1888]
Epoch [156/200] complete. Average Loss: 3.9778
Epoch [157/200]: 100%| | 400/400 [00:04<00:00, 90.49it/s, loss=1.7011]
Epoch [157/200] complete. Average Loss: 2.5212
Epoch [158/200]: 100%| | 400/400 [00:04<00:00, 89.79it/s, loss=1.2236]
Epoch [158/200] complete. Average Loss: 2.0538
Epoch [159/200]: 100%| | 400/400 [00:04<00:00, 89.37it/s, loss=0.5177]
Epoch [159/200] complete. Average Loss: 2.3207
Epoch [160/200]: 100%| | 400/400 [00:04<00:00, 90.32it/s, loss=1.7182]
Epoch [160/200] complete. Average Loss: 1.9824
Epoch [161/200]: 100%| | 400/400 [00:04<00:00, 88.99it/s, loss=0.1825]
Epoch [161/200] complete. Average Loss: 1.6815
Epoch [162/200]: 100%| | 400/400 [00:04<00:00, 89.01it/s, loss=1.8770]

Epoch [162/200] complete. Average Loss: 2.7068
Epoch [163/200]: 100%| | 400/400 [00:04<00:00, 90.74it/s, loss=0.6477]
Epoch [163/200] complete. Average Loss: 2.3915
Epoch [164/200]: 100%| | 400/400 [00:04<00:00, 89.52it/s, loss=5.0127]
Epoch [164/200] complete. Average Loss: 2.6983
Epoch [165/200]: 100%| | 400/400 [00:04<00:00, 89.84it/s, loss=0.5740]
Epoch [165/200] complete. Average Loss: 2.7980
Epoch [166/200]: 100%| | 400/400 [00:04<00:00, 90.01it/s, loss=0.3526]
Epoch [166/200] complete. Average Loss: 2.5075
Epoch [167/200]: 100%| | 400/400 [00:04<00:00, 88.90it/s, loss=1.2053]
Epoch [167/200] complete. Average Loss: 1.8428
Epoch [168/200]: 100%| | 400/400 [00:04<00:00, 89.42it/s, loss=4.4922]
Epoch [168/200] complete. Average Loss: 2.6136
Epoch [169/200]: 100%| | 400/400 [00:04<00:00, 89.81it/s, loss=1.6222]
Epoch [169/200] complete. Average Loss: 4.5033
Epoch [170/200]: 100%| | 400/400 [00:04<00:00, 90.36it/s, loss=1.3389]
Epoch [170/200] complete. Average Loss: 2.4715
Epoch [171/200]: 100%| | 400/400 [00:04<00:00, 89.51it/s, loss=0.4876]
Epoch [171/200] complete. Average Loss: 1.8097
Epoch [172/200]: 100%| | 400/400 [00:04<00:00, 89.58it/s, loss=1.2828]
Epoch [172/200] complete. Average Loss: 2.2909
Epoch [173/200]: 100%| | 400/400 [00:04<00:00, 89.42it/s, loss=2.3107]
Epoch [173/200] complete. Average Loss: 4.7080
Epoch [174/200]: 100%| | 400/400 [00:04<00:00, 89.35it/s, loss=6.1471]
Epoch [174/200] complete. Average Loss: 2.1998
Epoch [175/200]: 100%| | 400/400 [00:05<00:00, 77.50it/s, loss=0.4220]
Epoch [175/200] complete. Average Loss: 1.7727
Epoch [176/200]: 100%| | 400/400 [00:05<00:00, 71.46it/s, loss=2.4677]
Epoch [176/200] complete. Average Loss: 2.3740
Epoch [177/200]: 100%| | 400/400 [00:04<00:00, 88.84it/s, loss=1.6695]
Epoch [177/200] complete. Average Loss: 3.0479
Epoch [178/200]: 100%| | 400/400 [00:04<00:00, 90.73it/s, loss=1.3456]

Epoch [178/200] complete. Average Loss: 1.7986
Epoch [179/200]: 100%| | 400/400 [00:04<00:00, 90.76it/s, loss=4.2546]
Epoch [179/200] complete. Average Loss: 2.0081
Epoch [180/200]: 100%| | 400/400 [00:04<00:00, 89.86it/s, loss=1.3469]
Epoch [180/200] complete. Average Loss: 1.5515
Epoch [181/200]: 100%| | 400/400 [00:04<00:00, 89.32it/s, loss=4.2474]
Epoch [181/200] complete. Average Loss: 1.9282
Epoch [182/200]: 100%| | 400/400 [00:04<00:00, 89.19it/s, loss=1.7856]
Epoch [182/200] complete. Average Loss: 2.6220
Epoch [183/200]: 100%| | 400/400 [00:04<00:00, 90.78it/s, loss=1.4865]
Epoch [183/200] complete. Average Loss: 1.9917
Epoch [184/200]: 100%| | 400/400 [00:04<00:00, 88.33it/s, loss=0.8548]
Epoch [184/200] complete. Average Loss: 1.5371
Epoch [185/200]: 100%| | 400/400 [00:04<00:00, 88.73it/s, loss=1.1304]
Epoch [185/200] complete. Average Loss: 2.0467
Epoch [186/200]: 100%| | 400/400 [00:04<00:00, 88.58it/s, loss=3.9816]
Epoch [186/200] complete. Average Loss: 2.2548
Epoch [187/200]: 100%| | 400/400 [00:04<00:00, 90.07it/s, loss=0.6957]
Epoch [187/200] complete. Average Loss: 3.6512
Epoch [188/200]: 100%| | 400/400 [00:04<00:00, 89.03it/s, loss=11.1534]
Epoch [188/200] complete. Average Loss: 5.8824
Epoch [189/200]: 100%| | 400/400 [00:04<00:00, 90.32it/s, loss=4.3997]
Epoch [189/200] complete. Average Loss: 3.3749
Epoch [190/200]: 100%| | 400/400 [00:04<00:00, 90.54it/s, loss=0.7348]
Epoch [190/200] complete. Average Loss: 2.6711
Epoch [191/200]: 100%| | 400/400 [00:04<00:00, 89.47it/s, loss=0.6253]
Epoch [191/200] complete. Average Loss: 1.7912
Epoch [192/200]: 100%| | 400/400 [00:04<00:00, 89.22it/s, loss=0.3033]
Epoch [192/200] complete. Average Loss: 1.5372
Epoch [193/200]: 100%| | 400/400 [00:04<00:00, 88.55it/s, loss=2.9175]
Epoch [193/200] complete. Average Loss: 1.5266
Epoch [194/200]: 100%| | 400/400 [00:04<00:00, 90.19it/s, loss=1.5737]

```

Epoch [194/200] complete. Average Loss: 1.8722
Epoch [195/200]: 100%|      | 400/400 [00:04<00:00, 90.14it/s, loss=0.4020]
Epoch [195/200] complete. Average Loss: 2.7419
Epoch [196/200]: 100%|      | 400/400 [00:04<00:00, 89.62it/s, loss=0.3752]
Epoch [196/200] complete. Average Loss: 2.1298
Epoch [197/200]: 100%|      | 400/400 [00:04<00:00, 89.75it/s, loss=0.7985]
Epoch [197/200] complete. Average Loss: 3.1612
Epoch [198/200]: 100%|      | 400/400 [00:04<00:00, 90.04it/s, loss=1.8100]
Epoch [198/200] complete. Average Loss: 4.7666
Epoch [199/200]: 100%|      | 400/400 [00:04<00:00, 89.42it/s, loss=0.4657]
Epoch [199/200] complete. Average Loss: 2.7121
Epoch [200/200]: 100%|      | 400/400 [00:04<00:00, 89.17it/s, loss=0.3006]
C:\Users\Amogh\AppData\Local\Temp\ipykernel_29396\3180219614.py:4: UserWarning:
To copy construct from a tensor, it is recommended to use
sourceTensor.detach().clone() or
sourceTensor.detach().clone().requires_grad_(True), rather than
torch.tensor(sourceTensor).
    self.center = torch.tensor(center, dtype=torch.float32)
C:\Users\Amogh\AppData\Local\Temp\ipykernel_29396\3180219614.py:5: UserWarning:
To copy construct from a tensor, it is recommended to use
sourceTensor.detach().clone() or
sourceTensor.detach().clone().requires_grad_(True), rather than
torch.tensor(sourceTensor).
    self.lengths = torch.tensor(dims, dtype=torch.float32)
C:\Users\Amogh\AppData\Local\Temp\ipykernel_29396\3180219614.py:6: UserWarning:
To copy construct from a tensor, it is recommended to use
sourceTensor.detach().clone() or
sourceTensor.detach().clone().requires_grad_(True), rather than
torch.tensor(sourceTensor).
    self.quaternion = torch.tensor(quaternion, dtype=torch.float32)
C:\Users\Amogh\AppData\Local\Temp\ipykernel_29396\3180219614.py:29: UserWarning:
To copy construct from a tensor, it is recommended to use
sourceTensor.detach().clone() or
sourceTensor.detach().clone().requires_grad_(True), rather than
torch.tensor(sourceTensor).
    self.plane = torch.tensor(plane_normal, dtype=torch.float32)
C:\Users\Amogh\AppData\Local\Temp\ipykernel_29396\3180219614.py:30: UserWarning:
To copy construct from a tensor, it is recommended to use
sourceTensor.detach().clone() or
sourceTensor.detach().clone().requires_grad_(True), rather than
torch.tensor(sourceTensor).
    self.point_on_plane = torch.tensor(point_on_plane, dtype=torch.float32)

```

Epoch [200/200] complete. Average Loss: 2.2756
Training finished.
Loss chart saved to 'training_loss_chart.png'

VISUALIZING A RECONSTRUCTION FROM THE TEST SET ###
#####

--- Original Shape Tree ---

=====
DSL Object Structure
=====

```
Union
  Union
    Box(label_id=1, center=[-0.009999999776482582, 0.019999999552965164,
0.07000000029802322], dims=[0.8199999928474426, 0.14000000059604645,
0.8399999737739563], quat=[0.0, 0.0, 0.0, 1.0])
    Symmetry(Reflection) across plane with normal=[1.0, -0.0, -0.0] at
point=[-0.009999999776482582, -0.46000000834465027, 0.4000000059604645]
  Union
    Box(label_id=2, center=[-0.33000001311302185,
-0.46000000834465027, 0.4000000059604645], dims=[0.17000000178813934,
0.8199999928474426, 0.1599999964237213], quat=[-0.0, -0.0, 0.0, 1.0])
    Box(label_id=2, center=[-0.33000001311302185,
-0.46000000834465027, -0.1000000149011612], dims=[0.17000000178813934,
0.8199999928474426, 0.1599999964237213], quat=[-0.0, -0.0, -0.0, 1.0])
    Box(label_id=0, center=[-0.009999999776482582, 0.3799999952316284,
-0.2199999988079071], dims=[0.8199999928474426, 0.6600000262260437,
0.25999999046325684], quat=[0.0, 0.0, 0.0, 1.0])
=====
```

--- STARTING RECONSTRUCTION WITH DEBUG TRACE ---

```
[DEBUG] Model PREDICTED node type: Union
[DEBUG] Recursing into 2 children for Union...
[DEBUG] Model PREDICTED node type: Union
[DEBUG] Recursing into 2 children for Union...
[DEBUG] Model PREDICTED node type: Box
[DEBUG] SUCCESS: Reached a valid leaf node (Box).
[DEBUG] Model PREDICTED node type: SymRef
[DEBUG] Recursing into 1 child for Symmetry...
[DEBUG] Model PREDICTED node type: Union
[DEBUG] Recursing into 2 children for Union...
[DEBUG] Model PREDICTED node type: Box
[DEBUG] SUCCESS: Reached a valid leaf node (Box).
[DEBUG] Model PREDICTED node type: Box
```

```

[DEBUG] SUCCESS: Reached a valid leaf node (Box).
[DEBUG] SUCCESS: Reconstructed Union with 2 children.
[DEBUG] SUCCESS: Reconstructed SymRef with 1 child.
[DEBUG] SUCCESS: Reconstructed Union with 2 children.
[DEBUG] Model PREDICTED node type: Box
[DEBUG] SUCCESS: Reached a valid leaf node (Box).
[DEBUG] SUCCESS: Reconstructed Union with 2 children.

```

--- FINISHED RECONSTRUCTION ATTEMPT ---

--- Reconstructed Shape Tree (after training) ---

=====

DSL Object Structure

=====

Union

Union

```

Box(label_id=0, center=[-0.05000000074505806, -0.029999999329447746,
0.10000000149011612], dims=[0.7699999809265137, 0.3499999940395355,
0.6800000071525574], quat=[-0.009999999776482582, 0.009999999776482582,
0.009999999776482582, 0.9700000286102295])

```

```

Symmetry(Reflection) across plane with normal=[0.9800000190734863,
-0.05000000074505806, 0.03999999910593033] at point=[0.009999999776482582,
-0.36000001430511475, 0.05000000074505806]

```

Union

```

Box(label_id=1, center=[-0.27000001072883606,
-0.5099999904632568, 0.029999999329447746], dims=[0.14000000059604645,
0.4399999976158142, 0.1599999964237213], quat=[0.0, -0.009999999776482582,
0.009999999776482582, 0.9700000286102295])

```

```

Box(label_id=1, center=[-0.30000001192092896,
-0.490000000953674316, 0.07000000029802322], dims=[0.05999999865889549,
0.4399999976158142, 0.14000000059604645], quat=[0.009999999776482582,
-0.019999999552965164, 0.009999999776482582, 0.9700000286102295])

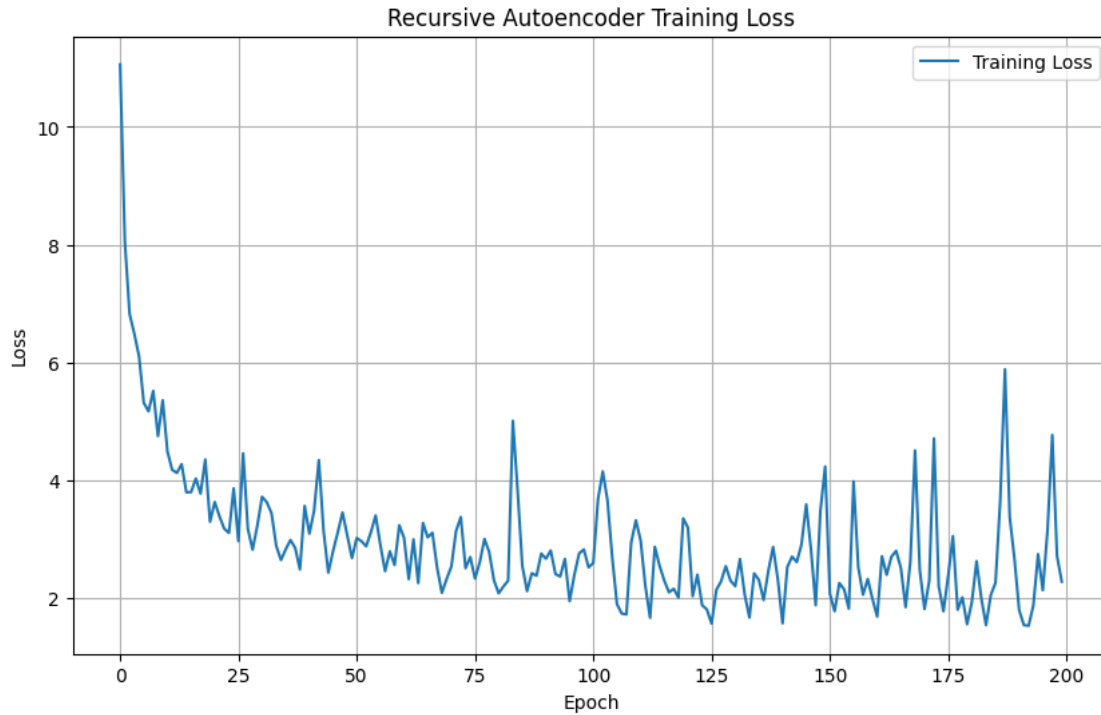
```

```

Box(label_id=0, center=[0.029999999329447746, 0.3799999952316284,
-0.17000000178813934], dims=[0.6100000143051147, 0.8399999737739563,
0.07000000029802322], quat=[0.03999999910593033, 0.05999999865889549,
0.019999999552965164, 1.059999942779541])

```

=====



```
[30]: torch.save(autoencoder.state_dict(), 'recursive_autoencoder.pth')
print("Model saved to recursive_autoencoder.pth")
```

Model saved to recursive_autoencoder.pth

```
[63]: import torch
import torch.nn as nn
from sklearn.model_selection import train_test_split
import random
import os
from tqdm import tqdm
from IPython.display import display

# --- Re-importing necessary components ---
# These are the model classes that need to be defined to load the weights.
class BoxEncoder(nn.Module):
    def __init__(self, hidden_dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(11, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim)
        )
    def forward(self, params): return self.net(params)
```

```

class CompositionEncoder(nn.Module):
    def __init__(self, hidden_dim):
        super().__init__()
        self.nets = nn.ModuleDict({
            "Union": nn.Linear(2 * hidden_dim, hidden_dim),
            "SymRef": nn.Linear(hidden_dim + 6, hidden_dim),
            "SymRot": nn.Linear(hidden_dim + 7, hidden_dim),
            "SymTrans": nn.Linear(hidden_dim + 4, hidden_dim),
        })
    def forward(self, node_type_str, params, child_features):
        combined_child_features = torch.cat(child_features, dim=-1)
        if params is not None:
            model_input = torch.cat([combined_child_features, params], dim=-1)
        else:
            model_input = combined_child_features
        return self.nets[node_type_str](model_input)

class TreeEncoder(nn.Module):
    def __init__(self, hidden_dim, latent_dim):
        super().__init__()
        self.box_encoder = BoxEncoder(hidden_dim)
        self.composition_encoder = CompositionEncoder(hidden_dim)
        self.root_to_latent = nn.Linear(hidden_dim, latent_dim)
    def _forward_recursive(self, v_node):
        node_type_str = IDX_TO_NODE_TYPE[v_node["type"].item()]
        if node_type_str == "Box": return self.box_encoder(v_node["params"])
        child_features = [self._forward_recursive(c) for c in v_node["children"]]
        params = v_node.get("params")
        return self.composition_encoder(node_type_str, params, child_features)
    def forward(self, v_tree):
        root_hidden_vec = self._forward_recursive(v_tree)
        z = self.root_to_latent(root_hidden_vec)
        return z

class NodeDecoder(nn.Module):
    def __init__(self, hidden_dim):
        super().__init__()
        self.net = nn.Sequential(nn.Linear(hidden_dim, hidden_dim), nn.ReLU())
        self.type_head = nn.Linear(hidden_dim, len(NODE_TYPE_TO_IDX))
        self.param_head = nn.Linear(hidden_dim, max([11, 6, 7, 4])) # Use max of
        all param sizes
        self.child_head = nn.Linear(hidden_dim, 2 * hidden_dim)
    def forward(self, parent_feature):
        h = self.net(parent_feature)
        return self.type_head(h), self.param_head(h), self.child_head(h)

```

```

class TreeDecoder(nn.Module):
    def __init__(self, latent_dim, hidden_dim):
        super().__init__()
        self.latent_to_hidden = nn.Linear(latent_dim, hidden_dim)
        self.node_decoder = NodeDecoder(hidden_dim)

    def _forward_recursive(self, parent_feature, gt_node):
        type_logits, param_preds, child_features = self.
        ↪node_decoder(parent_feature)
        reconstructed_node = {"type_logits": type_logits, "params":
        ↪param_preds, "children": []}
        if gt_node["children"]:
            child1_feature, child2_feature = torch.chunk(child_features, 2,
        ↪dim=-1)
            child_features_split = [child1_feature, child2_feature]
            for i, gt_child_node in enumerate(gt_node["children"]):
                reconstructed_child = self.
        ↪_forward_recursive(child_features_split[i], gt_child_node)
                reconstructed_node["children"].append(reconstructed_child)
            return reconstructed_node
    def forward(self, z, gt_tree):
        initial_hidden = self.latent_to_hidden(z)
        return self._forward_recursive(initial_hidden, gt_tree)

class RecursiveAutoencoder(nn.Module):
    def __init__(self, hidden_dim, latent_dim):
        super().__init__()
        self.encoder = TreeEncoder(hidden_dim, latent_dim)
        self.decoder = TreeDecoder(latent_dim, hidden_dim)

    def forward(self, v_tree):
        z = self.encoder(v_tree)
        reconstructed_tree = self.decoder(z, v_tree)
        return reconstructed_tree

# --- Main execution to load the model and reconstruct a shape ---
if __name__ == "__main__":
    # --- Configuration
    HIDDEN_DIM = 128
    LATENT_DIM = 64
    DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    NODE_TYPE_TO_IDX = {"Box": 0, "Union": 1, "SymRef": 2, "SymRot": 3,
    ↪"SymTrans": 4, "<EOS>": 5}
    IDX_TO_NODE_TYPE = {v: k for k, v in NODE_TYPE_TO_IDX.items()}
    # --- Load the saved model
    autoencoder = RecursiveAutoencoder(HIDDEN_DIM, LATENT_DIM).to(DEVICE)
    try:

```

```

        autoencoder.load_state_dict(torch.load('recursive_autoencoder.pth',
↪map_location=DEVICE))
        print("Model loaded successfully from recursive_autoencoder.pth")
    except FileNotFoundError:
        print("Error: The model file 'recursive_autoencoder.pth' was not found.
↪Please save the model first.")
        # Exit to prevent further errors
        exit()
    # Set the model to evaluation mode
    autoencoder.eval()
    # --- Prepare a shape for reconstruction
    # Assumes 'dataset/Chair' directory exists
    dataset_dir = "dataset/Chair"
    all_json_files = [os.path.join(dataset_dir, f) for f in os.
↪listdir(dataset_dir) if f.endswith('.json')]
    if not all_json_files:
        print("Error: No chair files found in the dataset directory.")
        exit()
    # Pick a random file from the test set
    full_dataset = []
    for filepath in tqdm(all_json_files[:100], desc="Loading Shapes"):
        with open(filepath, 'r') as f:
            json_string = f.read()
            dsl_tree = parse_json_to_dsl(json_string)
            vectorized_tree = vectorize_tree(dsl_tree, DEVICE)
            full_dataset.append((dsl_tree, vectorized_tree))
    _, test_dataset = train_test_split(full_dataset, test_size=0.2,
↪random_state=42)
    original_dsl_shape, vectorized_shape = random.choice(test_dataset)
    # --- Reconstruct the shape
    with torch.no_grad():
        reconstruction_output = autoencoder(vectorized_shape)
        final_reconstructed_shape = devectorize_tree_debug(reconstruction_output)
    # --- Display the results
    print("\n--- Original Shape ---")
    print_dsl(original_dsl_shape)

    print("\n--- Reconstructed Shape ---")
    if final_reconstructed_shape:
        print_dsl(final_reconstructed_shape)
        # Assuming plot_dsl_with_k3d is available from your other notebook cell
        plot_dsl_with_k3d(original_dsl_shape)
        plot_dsl_with_k3d(final_reconstructed_shape)
    else:
        print("Reconstruction failed.")

```

Model loaded successfully from recursive_autoencoder.pth

```

Loading Shapes: 100%|      | 100/100 [00:00<00:00, 565.70it/s]
C:\Users\Amogh\AppData\Local\Temp\ipykernel_29396\3180219614.py:4: UserWarning:
To copy construct from a tensor, it is recommended to use
sourceTensor.detach().clone() or
sourceTensor.detach().clone().requires_grad_(True), rather than
torch.tensor(sourceTensor).
    self.center = torch.tensor(center, dtype=torch.float32)
C:\Users\Amogh\AppData\Local\Temp\ipykernel_29396\3180219614.py:5: UserWarning:
To copy construct from a tensor, it is recommended to use
sourceTensor.detach().clone() or
sourceTensor.detach().clone().requires_grad_(True), rather than
torch.tensor(sourceTensor).
    self.lengths = torch.tensor(dims, dtype=torch.float32)
C:\Users\Amogh\AppData\Local\Temp\ipykernel_29396\3180219614.py:6: UserWarning:
To copy construct from a tensor, it is recommended to use
sourceTensor.detach().clone() or
sourceTensor.detach().clone().requires_grad_(True), rather than
torch.tensor(sourceTensor).
    self.quaternion = torch.tensor(quaternion, dtype=torch.float32)

[DEBUG] Model PREDICTED node type: Union
[DEBUG] Recursing into 2 children for Union...
[DEBUG] Model PREDICTED node type: Union
[DEBUG] Recursing into 2 children for Union...
[DEBUG] Model PREDICTED node type: Box
[DEBUG] SUCCESS: Reached a valid leaf node (Box).
[DEBUG] Model PREDICTED node type: Box
[DEBUG] SUCCESS: Reached a valid leaf node (Box).
[DEBUG] SUCCESS: Reconstructed Union with 2 children.
[DEBUG] Model PREDICTED node type: Box
[DEBUG] SUCCESS: Reached a valid leaf node (Box).
[DEBUG] SUCCESS: Reconstructed Union with 2 children.

--- Original Shape ---

=====
DSL Object Structure
=====
Union
    Union
        Box(label_id=1, center=[0.00999999776482582, 0.01999999552965164,
0.05000000074505806], dims=[1.190000057220459, 0.46000000834465027,
1.2200000286102295], quat=[0.05999999865889549, -0.36000001430511475,
-0.009999999776482582, 0.9300000071525574])
        Box(label_id=2, center=[-0.0, -0.33000001311302185,
0.029999999329447746], dims=[1.149999976158142, 0.3499999940395355,
1.2100000381469727], quat=[0.0, 0.0, 0.0, 1.0])
        Box(label_id=0, center=[-0.0, 0.4699999988079071, -0.4000000059604645],

```

```

dims=[1.2000000476837158, 0.8100000023841858, 0.5699999928474426],
quat=[-0.23999999463558197, 0.009999999776482582, -0.009999999776482582,
0.9700000286102295])
=====

```

--- Reconstructed Shape ---

```

=====
DSL Object Structure
=====

```

```

Union
  Union
    Box(label_id=0, center=[-0.019999999552965164, 0.15000000596046448,
-0.019999999552965164], dims=[0.9599999785423279, 0.46000000834465027,
0.6700000166893005], quat=[0.0, 0.009999999776482582, 0.009999999776482582,
0.9700000286102295])
    Box(label_id=1, center=[0.05000000074505806, -0.449999998807907104,
0.0], dims=[0.8899999856948853, 0.23999999463558197, 0.8700000047683716],
quat=[0.029999999329447746, -0.05999999865889549, 0.03999999910593033,
1.0299999713897705])
    Box(label_id=0, center=[-0.03999999910593033, 0.3700000047683716,
-0.209999999344348907], dims=[0.9399999976158142, 0.8600000143051147,
0.3499999940395355], quat=[-0.05999999865889549, -0.05999999865889549,
0.03999999910593033, 0.9800000190734863])
=====

```

Expanding DSL tree for visualization...
Found 3 total boxes after expansion.

Output()

Expanding DSL tree for visualization...
Found 3 total boxes after expansion.

Output()

```

[ ]: 
[ ]: 
[ ]: 
[ ]: 
[ ]: 

```